Diffusion Models

GenAI and Modern Deep Learning

Stable Diffusion

Generated by Stable Diffusion

Takes in text prompts, generates images

OpenAI’s DALL·E 2 is another diffusion model that does this too

Let’s start with simpler diffusion models that don’t involve text

Just train on images

Denoising Diffusion Probabilistic Models (Ho 2020)

Generate images by iteratively denoising

The forward process (noising):

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_T$$

Slowly turn data distribution $x_0 \sim p(x)$ into noise $x_T \sim N(0, \sigma^2 I)$

x_0 x_1 x_T

Denoising Diffusion Probabilistic Models (Ho 2020)

The reverse process (denoising):

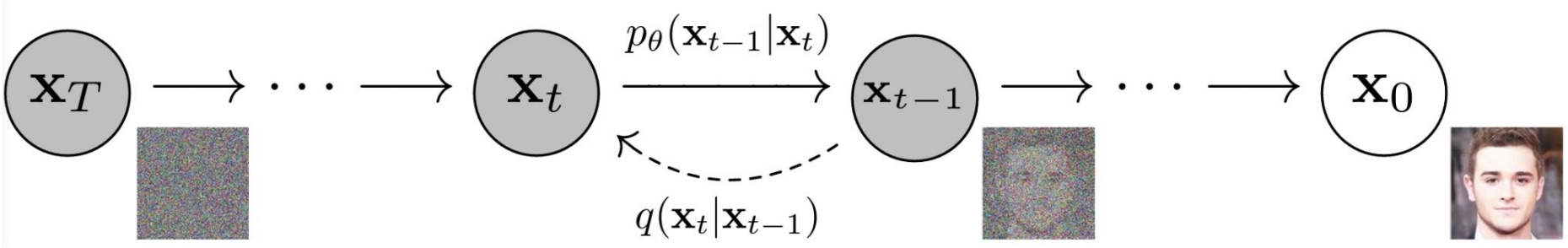
$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0$$

Sample from noise distribution $x_T \sim N(0, \sigma^2 I)$

Reverse noising process to generate sample from data distribution $x_0 \sim p(x)$

x_0 x_1 x_T

Denoising Diffusion Probabilistic Models (Ho 2020)



Denoising Diffusion Probabilistic Models (Ho 2020)

The forward process does not require any learning, adding noise is easy.

Define $q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$

Variance schedule $\beta_1, \ldots, \beta_T$ increases linearly (e.g. from 0.0001 to 0.8)

Denoising Diffusion Probabilistic Models (Ho 2020)

The forward process does not require any learning, adding noise is easy.

Define $q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$

Variance schedule $\beta_1, \ldots, \beta_T$ increases linearly (e.g. from 0.0001 to 0.8)

Why need this coefficient?

Want to sample from N(0, I), so mean should be close to zero for high t.

Low t: want to sample close to image, mean should be close to image.

Mean should interpolate between zero and target image

Denoising Diffusion Probabilistic Models (Ho 2020)

The forward process does not require any learning, adding noise is easy.

Don’t need to add noise step by step, can do it all at once.

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I})$$

Let $\alpha_t := 1 - \beta_t \qquad \bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$

Denoising Diffusion Probabilistic Models (Ho 2020)

Going backwards is harder, requires a denoiser which is the diffusion model

Model each backwards conditional by a Gaussian, estimate mean and covariance

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Need to fit these conditional distributions to the data

Let $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ and $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)$ depend on the output of a neural network

Denoising Diffusion Probabilistic Models (Ho 2020)

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Idea: train a neural network to output $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ and $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)$

In practice, let $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)$ be constant. Learning it makes training unstable

Just learn $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$

Could have neural network output $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ directly

Empirically, works best to predict noise $\boldsymbol{\epsilon}$ instead

Denoising Diffusion Probabilistic Models (Ho 2020)

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

Empirically, works best to predict noise $\boldsymbol{\epsilon}$ instead

Let $\boldsymbol{\epsilon}_\theta$ be the output of the neural network, predicts $\boldsymbol{\epsilon}$

Network takes in noisy image and timestep t

If we have noisy image and predicted noise, subtract predicted noise to get image

Diffusion as a VAE

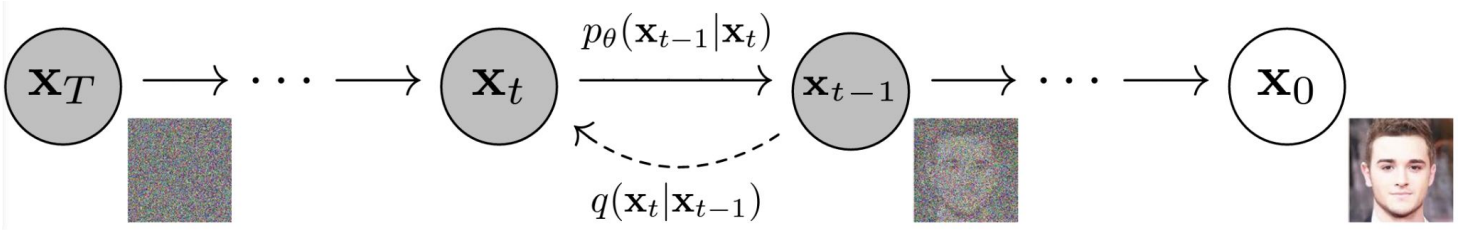
View noise process as VAE encoder

View denoising process as VAE decoder

Can compute analogous VAE loss (variational lower bound)

It ends up looking really messy. In practice, do something simpler.

MSE denoising loss



Denoising Diffusion Probabilistic Models (Ho 2020)

Can use fixed variance in practice, don't learn it

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Training image

MSE loss
Predict noise

Noised image

Denoising Diffusion Probabilistic Models (Ho 2020)

The original DDPM model was trained on small image datasets like CIFAR-10.

Ok so imagine we trained a diffusion model $\boldsymbol{\epsilon}_\theta$

How do we generate an image?

Recall, we parameterized each backwards conditional by a Gaussian.

Sample from those Gaussians one by one

Sampling from Gaussian with mean $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ is the same as sample mean-zero Gaussian noise and adding to $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$

Denoising Diffusion Probabilistic Models (Ho 2020)

The original DDPM model was trained on small image datasets like CIFAR-10.

Ok so imagine we trained a diffusion model ϵ_θ

How do we generate an image?

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Standard deviation of backwards conditional

Variance decreases per step

Mean of reverse Gaussian

Denoising Diffusion Probabilistic Models (Ho 2020)

Denoising Diffusion Probabilistic Models (Ho 2020)

Figure 3: LSUN Church samples. FID=7.89

Figure 4: LSUN Bedroom samples. FID=4.90

Score-Based Generative Models (Song 2019)

Came out around same time as DDPM. Different interpretation of diffusion.

But first, some background.

Stochastic gradient Langevin dynamics (SGLD):

Consider density $p(x)$ over images, and we want to sample.

Randomly sample initial point x_0

$x_t = x_{t-1} + \alpha \nabla_{x_t} \log p(x) + \sqrt{2\alpha}\epsilon_t$, where $\epsilon \sim N(0, 1)$

In the limit, this draws a sample from $p(x)$ as $t \to \infty, \epsilon \to 0$.

Score-Based Generative Models (Song 2019)

Buuut, we don't know $p(x)$. We'll deal with that in a sec

It gets worse.

If images lie on a low dimensional manifold, then they don't have a density.

Ideas:

Image plus tiny noise with variance σ^2 has density $q_\sigma(x)$, but gradient is tiny far away from image manifold

Image plus big noise has nonzero gradients far away from manifold, but samples are super noisy

Score-Based Generative Models (Song 2019)

Solution:

Start with image + big noise

Run SGLD using density $q_\sigma(x)$

Decrease noise

Keep running SGLD

…

…

Get a sample from $p(x)$

Score-Based Generative Models (Song 2019)

Other problem, we don't know $p(x)$ or similarly $q_\sigma(x)$

Can learn *score function* $s(x, \sigma) \approx \nabla_x \log q_\sigma(x)$

Can swap this score function out in SGLD formula.

How to learn score function?

Turns out (with some fancy math) that the following works:

$$\mathcal{L}(\theta, \sigma) = \frac{1}{2}\mathbb{E}_{p(x)}\mathbb{E}_{\tilde{x}\sim N(x,\sigma^2 I)}\|s_\theta(\tilde{x}, \sigma) + \frac{\tilde{x}-x}{\sigma^2}\|_2^2$$

But this again is just MSE predicting the noise.

Classifier-Free Guidance - How to condition on labels?

Add class label as input (or text) sometimes when training diffusion model

Algorithm 1 Joint training a diffusion model with classifier-free guidance

Require: p_{uncond}: probability of unconditional training

1: **repeat**
2: $(\mathbf{x}, \mathbf{c}) \sim p(\mathbf{x}, \mathbf{c})$ ▷ Sample data with conditioning from the dataset
3: $\mathbf{c} \leftarrow \varnothing$ with probability p_{uncond} ▷ Randomly discard conditioning to train unconditionally
4: $\lambda \sim p(\lambda)$ ▷ Sample log SNR value
5: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
6: $\mathbf{z}_\lambda = \alpha_\lambda \mathbf{x} + \sigma_\lambda \boldsymbol{\epsilon}$ ▷ Corrupt data to the sampled log SNR value
7: Take gradient step on $\nabla_\theta \left\| \boldsymbol{\epsilon}_\theta(\mathbf{z}_\lambda, \mathbf{c}) - \boldsymbol{\epsilon} \right\|^2$ ▷ Optimization of denoising model
8: **until** converged

signal-to-noise ratio
similar to choosing timestep
just different notation

Model learns to use class label

Classifier-Free Guidance

Generation time

Algorithm 2 Conditional sampling with classifier-free guidance

Require: w: guidance strength
Require: $\mathbf{c}$: conditioning information for conditional sampling
Require: $\lambda_1, \ldots, \lambda_T$: increasing log SNR sequence with $\lambda_1 = \lambda_{\min}, \lambda_T = \lambda_{\max}$

$\mathbf{z}_1 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
for $t = 1, \ldots, T$ **do**
  ▷ Form the classifier-free guided score at log SNR λ_t
  $\tilde{\boldsymbol{\epsilon}}_t = (1 + w)\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, \mathbf{c}) - w\boldsymbol{\epsilon}_\theta(\mathbf{z}_t)$
  ▷ Sampling step (could be replaced by another sampler, e.g. DDIM)
  $\tilde{\mathbf{x}}_t = (\mathbf{z}_t - \sigma_{\lambda_t}\tilde{\boldsymbol{\epsilon}}_t)/\alpha_{\lambda_t}$
  $\mathbf{z}_{t+1} \sim \mathcal{N}(\tilde{\boldsymbol{\mu}}_{\lambda_{t+1}|\lambda_t}(\mathbf{z}_t, \tilde{\mathbf{x}}_t), (\tilde{\sigma}^2_{\lambda_{t+1}|\lambda_t})^{1-v}(\sigma^2_{\lambda_t|\lambda_{t+1}})^v)$ if $t < T$ else $\mathbf{z}_{t+1} = \tilde{\mathbf{x}}_t$
end for
return $\mathbf{z}_{T+1}$

Only difference between CFG and what we saw before

The U-Net Architecture

Need an image-to-image denoising architecture

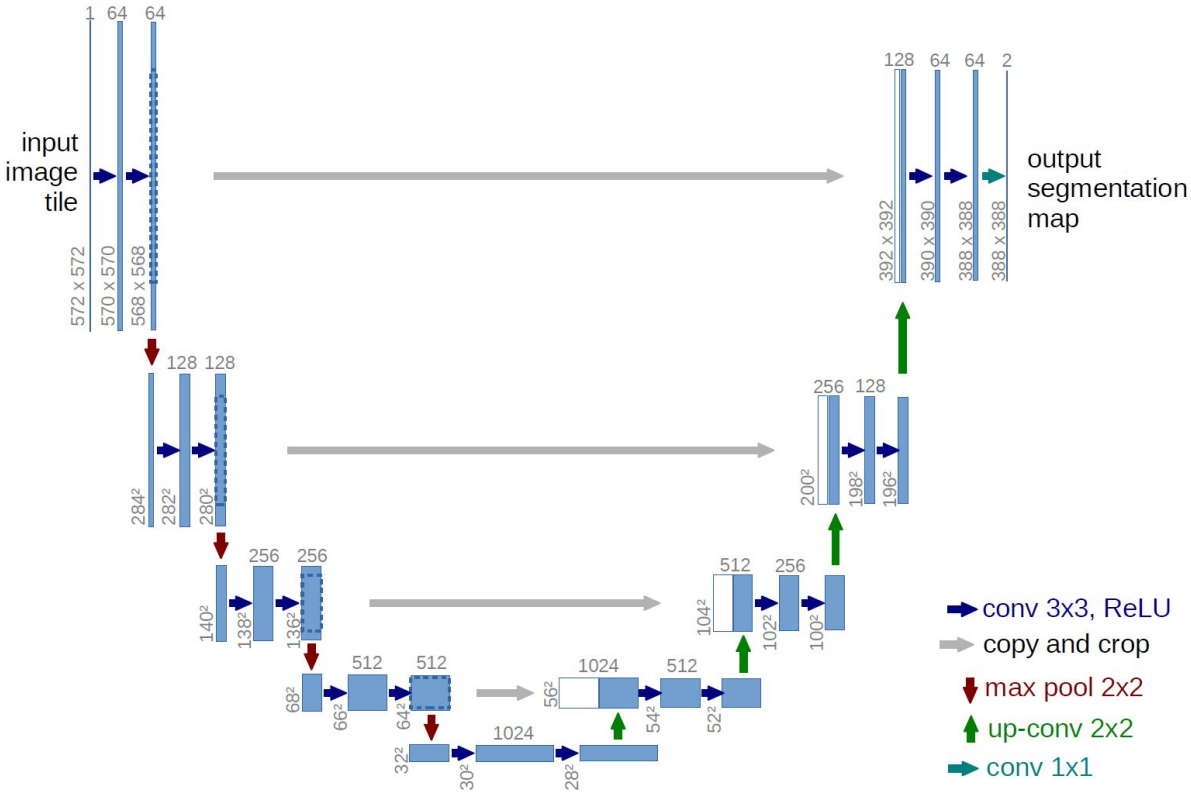
Problem: high res images are expensive

Solution: do lots of compute at lower spatial resolutions

Enables bigger models per latency

U-Nets - designed for segmentation, useful for all kinds of image-to-image tasks

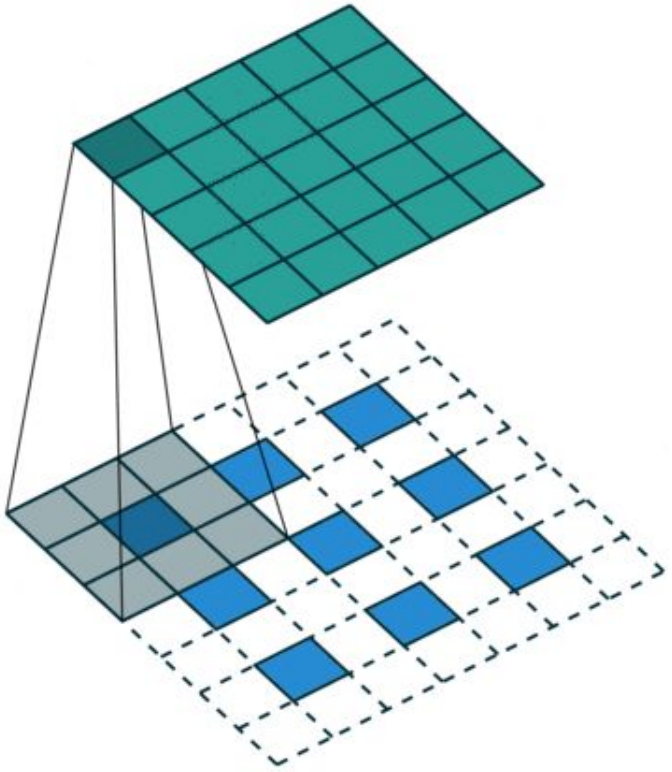
The U-Net Architecture



Note about Upsampling

Known as transposed convolutions or deconvolutions

Space out input with padding, increases spatial dimensions of output

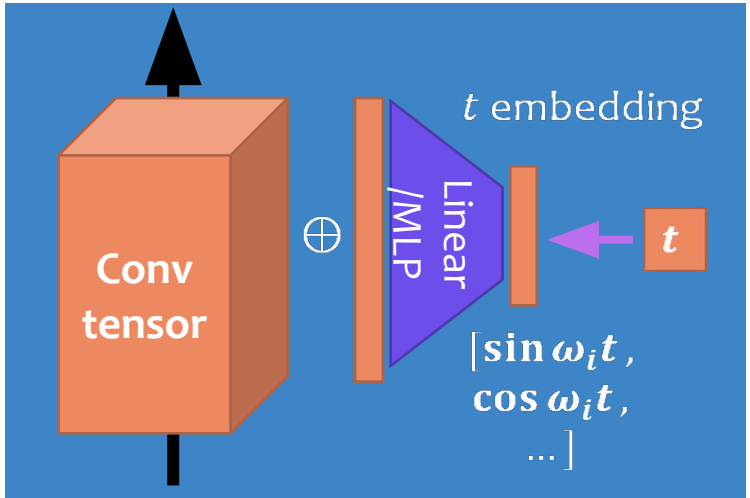


Adding in Time-Dependency

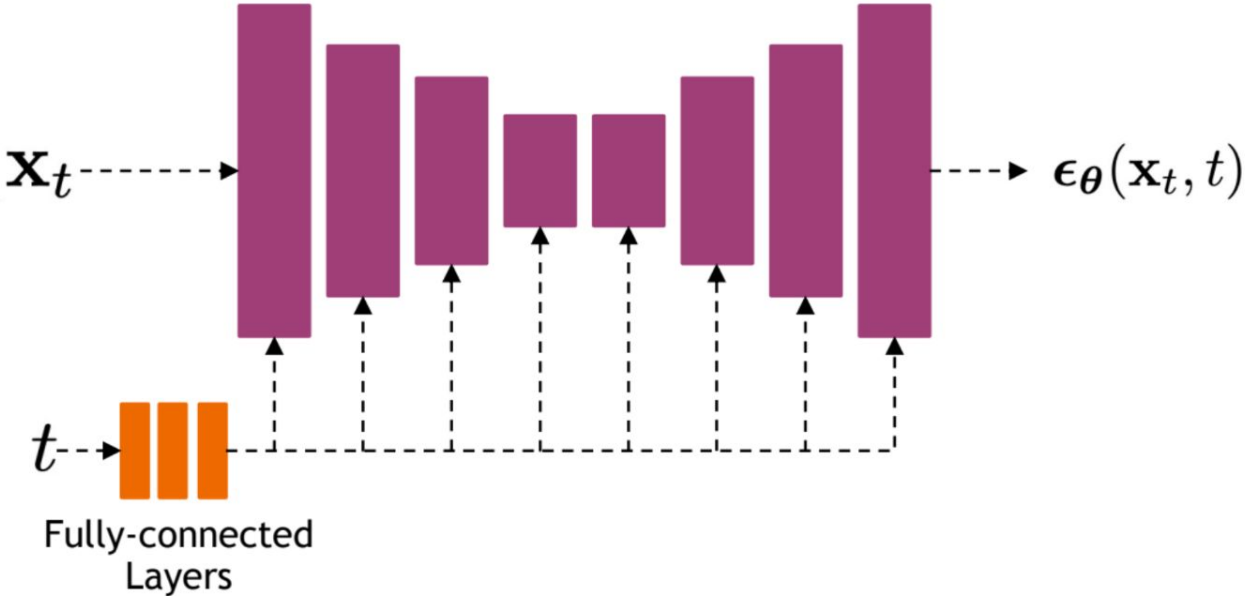
During generation, $\epsilon_\theta(x, t)$ takes in time as an input.

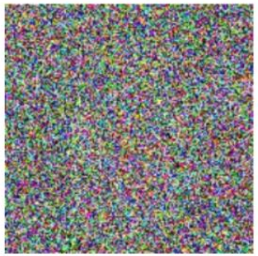
How to input time into the U-Net?

Can add a number to each conv channel or encode in normalization layers



Adding in Time-Dependency





The Stable Diffusion Architecture (Rombach 2022)

Latent diffusion - do diffusion in latent space instead of original space

- Train a VAE ahead of time and hold onto encoder and decoder
- VAEs map x -> z -> x, z has very small spatial dimensions
- Do diffusion in latent space z instead of x
- To train, do everything we previously did but in latent space
- Run each training image through encoder, get latent
- Learn to denoise latent
- Don’t need decoder for training
- To generate, sample Gaussian latent vector and denoise iteratively
- At the end, use decoder to map denoised Gaussian vector into an image
- Don’t need encoder for generation

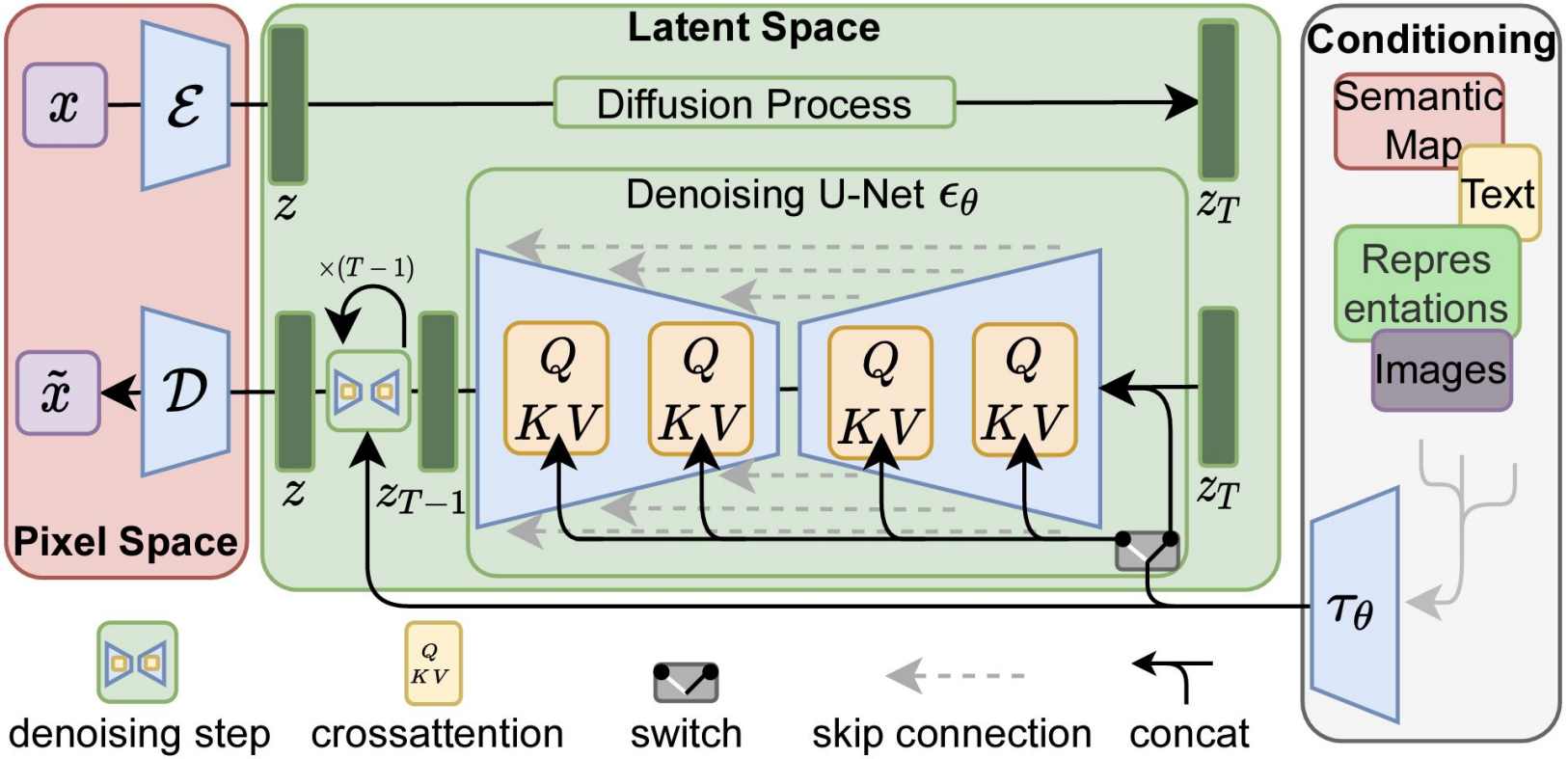
Latent diffusion is faster and scales better

The Stable Diffusion Architecture (Rombach 2022)

How to incorporate text conditioning?

- Extract features from prompt using CLIP or other text feature extractor
- Use cross-attention to input text features into U-Net
- U-Net denoises in latent space iteratively

The Stable Diffusion Architecture (Rombach 2022)

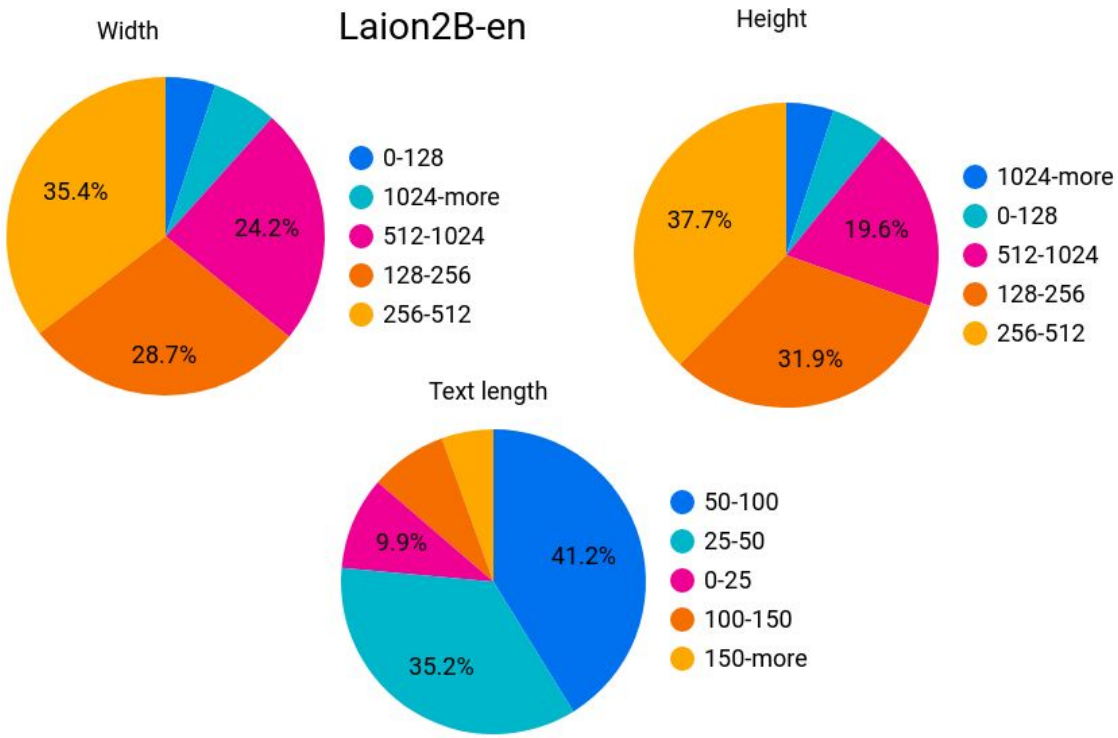


Summary of Training Procedure

- Train on captioned-image dataset. Pairs of (image, caption)
- Images are encoded through an encoder, which turns images into latent representations.
- Text prompts (captions) are encoded.
- The loss is a reconstruction objective between the noise that was added to the latent and the prediction made by the U-Net

Trick: pretrain on tons of data, fine-tune on higher quality data

Data for Training Stable Diffusion



Negative Prompts

Recall classifier-free guidance:

$$\tilde{\epsilon}_t = (1 + w)\epsilon_\theta(\mathbf{z}_t, \mathbf{c}) - w\epsilon_\theta(\mathbf{z}_t)$$

Pushes output far in class-conditional direction

Subtract out non class-conditional direction

Now, we're conditioning on text.

Replace second term with denoising conditioned on *"negative prompt"*

Pushes generated images away from negative prompt

Examples

Prompt: 2D Vector Illustration of a child with soccer ball Art for Sublimation, Design Art, Chrome Art, Painting and Stunning Artwork, Highly Detailed Digital Painting, Airbrush Art, Highly Detailed Digital Artwork, Dramatic Artwork, stained antique yellow copper paint, digital airbrush art, detailed by Mark Brooks, Chicano airbrush art, Swagger! snake Culture

Negative prompt: low resolution, low details

Parameters: Steps: 20, Sampler: Euler a, CFG scale: 7, Seed: 3177887799, Size: 1024x1024, Model hash: 70229e1d56, Model: SD XL

Choices to make when designing a diffusion model

How to design the best generative model is far from settled!

- Latent diffusion vs. image-space diffusion
- Cascade vs. all at once
- How to pretrain your image autoencoder for latent diffusion?
- One popular choice is VAE + patch-based adversarial loss, can also do VQ-GAN and just do vector quantization after latent space (build into decoder)
- Denoising architecture: CNN vs. transformer
- Training sets/stages: Pretrain, fine-tune on high-quality data? High-res data?

How to evaluate a diffusion model?

FID score - evaluates if model generates similar distribution to reference dataset

- Plug images from reference dataset into feature extractor (Inception v3)
- Fit Gaussian distribution to the features
- Do the same for images generated by diffusion model, get 2nd Gaussian
- Take distance between the two Gaussians (Fréchet distance), lower is better

CLIP score - evaluates if model generates appropriate image for text prompt

- CLIP contains image/text feature extractors with common feature space
- Input prompt to diffusion model, generate image
- Input prompt to CLIP text model, generated image to vision model
- Compare cosine similarity between features, higher is better

…

Hand inspection

Diffusion vs. GANs

Diffusion models generate amazing images

Training is stable, just denoising loss

GANs are way way faster!

One forward pass through the network

Lots of work on making diffusion models faster

Fine-Tuning Diffusion Models

Take small dataset of images you are interested in

e.g. particular movie characters

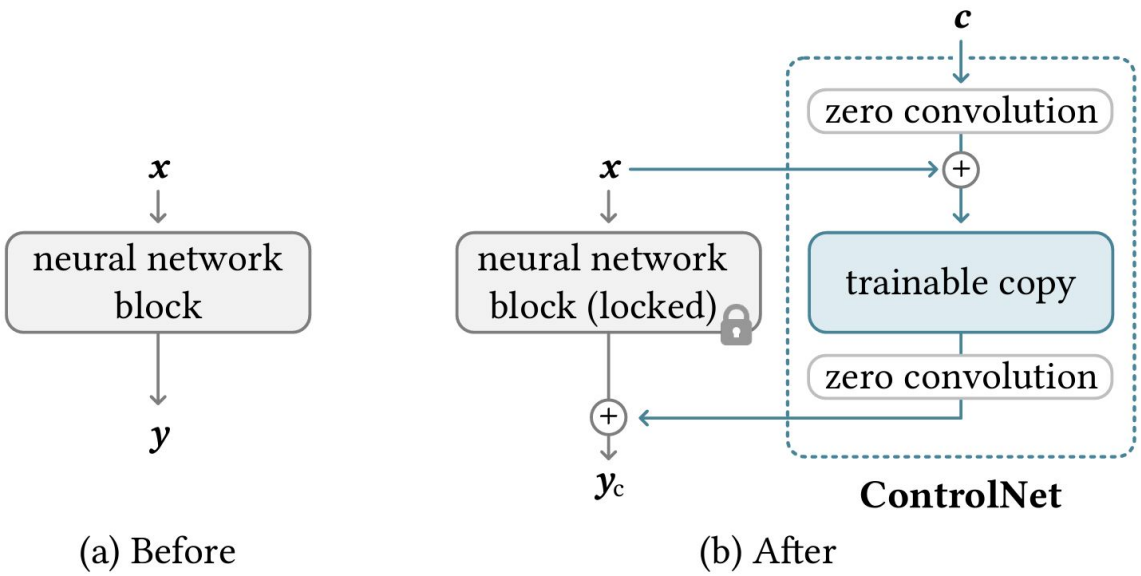
Fine-tune denoiser and text feature extractor

Can use LoRA (low-rank adaptors we saw for language models), end-to-end finetuning or any other adaptors

ControlNet (Zhang 2023)

Add new conditioning to existing diffusion model

Adapters initialized to start at pretrained model



ControlNet (Zhang 2023)

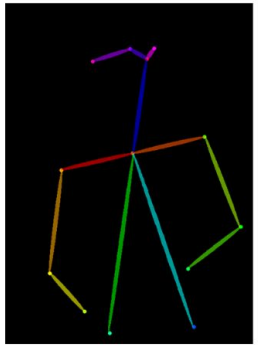
Originally used for spatial information

Input Canny edge

Default

"masterpiece of fairy tale, giant deer, golden antlers"

"..., quaint city Galic"



Input human pose

Default

"chef in kitchen"

"Lincoln statue"

Autoregressive Generative Models for Images (Sun 2024)

VQ-VAE or VQ-GAN can map images in and out of discrete space

Treat discrete tokens like text tokens

Training:

- Run image dataset through encoder, get sequence of tokens
- Train LLM-style transformer with causal language modeling objective

Generation:

- Generate sequence of tokens using transformer
- Use VQ-GAN decoder to transform into image

Autoregressive Generative Models for Images (Sun 2024)

Figure 1: **Image generation with vanilla autoregressive models**. We show samples from our class-conditional image (top row) and text-conditional image (bottom row) generation models.